



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری  
نام و نام خانوادگی: علیرضا کریمی



نوت ۹ آذر

## فصل چهارم: لایه *Network* بخش *Data – Plane* اسلاید ۶۲ تا ۸۰ [۱]

خب نمی‌توانیم که تمام روترها را در یک روز آپگرید کنیم و از *IPv4* به *IPv6* آپگرید کنیم. پس هیچ روزی موسوم به روز *flag day* نداریم که همزمان همه را ارتقا دهیم. اما این ارتقا را میتوانیم به صورت پله پله انجام بدهیم. اینگونه ممکن است همزمان یک *mix* از روترهای *IPv4* و *IPv6* در شبکه وجود داشته باشد. پس از آنجایی که این روترها هر کدوم هدر مخصوص به تکنولوژی خود را می‌فهمند، نیاز به روشی داریم که بتوان پیام را از میان مخلوطی از روترهای *IPv4* و *IPv6* منتقل کرد.

### روش *Tunneling*

در این روش، در صورتی که یک بسته دیتاگرام *IPv6* به روتری رسید روتر بعدی آن که تنها تکنولوژی *IPv4* را پشتیبانی میکند، آن بسته را به عنوان *payload* در یک بسته دیتاگرام *IPv4* قرار می‌دهیم و به آن روتر می‌دهیم. حال *source* و *destination* در هدر *IPv4* را برابر با دو روتری مرزی قرار می‌دهیم که بین آن‌ها مسیر *IPv4* قرار دارد و تمام روترهای آن مسیر از *IPv4* هستند. توجه کنید در خود هدرهای *IPv6* هدرهای *source* و *destination* داریم که باید همان مبدا و مقصد اولیه باشند. اما برای مسیرهای *IPv4*، در سر مسیر را که روترهای مرزی هستند و هر دو تکنولوژی *IPv4* و *IPv6* ساپورت میکنند قرار می‌دهیم و *source* و *destination* هدر بسته *IPv4* را این دو روتر مرزی قرار می‌دهیم و نه روترهای مبدا و مقصد. بسته *IPv6* هم در قالب *payload* در بسته *IPv4*، *encapsulate* شده است. یک فیلد *flow* هم در *IPv6* است که در هدر *IPv4* نیست. برای بسته‌های مرتب با هم.

### *Generalized forwarding*

در روش‌های مبتنی بر نرم افزار برای مسیر یابی *SDN*، *data plane* و *control plane* از هم جدا شدند، مثل روش سنتی نیست که همه روترها توابع پیچیده مسیر یابی را در خود داشته باشند، در عوض مسیریابی توسط یک سیستم کنترلر بیرونی و منطقاً متمرکز انجام میشود. *remote controller* این کنترلر در هر لحظه گراف شبکه را دارد.

طبق آن مسیر یابی میکند و اطلاعات لازم موسوم به *flow table* را که همان *forwarding table* تعمیم یافته و *generalized* است، به تابع *control agent* در هر روتر میدهد (این تابع ساده، راه ارتباطی میان کنترلر و روتر است) نکته مهم این *flow table* است که نسبت به حالت قبلی گفته شده در *forwarding table* که تنها میتواندست عمل *forwarding* را انجام دهد، اینجا کارهای متنوع بیشتری صورت میگیرد که به تفصیل به آن میپردازیم. یعنی بعد از *match* کردن اولیه، *action* های مختلفی علاوه بر فورواردینگ میتواند صورت بگیرد.

## OpenFlow

یک نوع پیاده سازی از همان مفهوم *generalized forwarding* است. این روش ۴ عمل اصلی را انجام میدهد:

۱. *match*: اینکه از روی هدرها و *pattern* های مربوطه و یک سری قوانین مربوط به هدرهای مختلف داده، آن‌ها را تطبیق دهد.
۲. *action*: اینکه بعد از تطبیق، بسته را *forward* کند یا *drop* کند یا *modify* کند و یا به کنترلر انتقالش دهد.
۳. *priority*: اینکه بین بسته‌های با یک الگوی هدر یکسان و *overlapping*، اولویت بگذارد.
۴. *counters*: اینکه یک سری اطلاعات آماری از بسته‌ها جمع‌آوری کند، مثلاً تعداد بایت‌ها یا بسته‌های عبوری و غیره

این الگوها طبق یک سری *rule* انجام میشود. مثلاً اگر *match* بر اساس آدرس *MAC* باشد، به عنوان *switch* در نظر گرفته میشود. *layer 2 destination – based forwarding*. اینجا احتمالاً پیام را به نوعی برودکست میکند که به آن *flood* میگویند

اگر طبق آدرس و پورت مقصد باشد (یا به علاوه آدرس و پورت مبدا)، به عنوان روتر.

اگر در *action*، آن پورت و آدرس مبدا را به پورت و آدرس مبدا دیگری مپ کند و *rewrite* کند، به عنوان *NAT*، و اگر هم جلوی یک سری پکت‌ها از پورت و آدرس خاص را بگیرد، به عنوان *firewall* در نظر گرفته می‌شود. که وظیفه *deny* یا *permit* را دارد.

## middlebox

یک باکس میانی در مسیر داده بین مبدا و مقصد، که میتواند یک تابع متفاوت را اجرا کند غیر از توابع روال و استاندارد. قابلیت برنامه ریزی دارد (مثلاً عمل‌های محلی در سطح سوئیچ)

شبکه‌های نرم افزار محور، این توانایی را میدهد که مدیریت پیکربندی کارآمدی که در مفهومی موسوم به *NFV* = *network function virtualization* امکان پذیر شده است، را داشته باشیم، چه در سطح *cloud*، چه خصوصی

چه عمومی.

میتوانیم جعبه‌های فایروال، لود بالانسر، NAT و غیره را میتوان در NFV با قابلیت برنامه ریزی شبکه، محاسبات و ذخیره سازی، پیاده سازی کرد.

*whitebox*

این نوع سوویچ‌ها، توسط SDN مدیریت میشوند و ISP ها میتوانند OS مربوطه و لازم را روی آن دانلود کند و قرار دهند. پس مبتنی بر پردازنده‌های همه منظوره هستند و همه منظوره هستند. و ارزانتر. پس ISP ها سوویچ‌های *whitebox* و SDN را در دیتاسنتر و NFV را در هسته به کار میگیرند و مقیاس پذیری را با صرف هزینه کم انجام میدهند.

از انواع معروف *middle box* ها:

- NAT در خانه و موسسات و شبکه‌های سلولی
- *firewall* ها و *IDS* ها *intrusion detection system* یا سیستم‌های تشخیص نفوذ در شرکت‌ها و موسسات و ISP ها
- *load balancer* در شبکه‌های سیار *mobile net* و ISP ها و شرکت‌ها و مراکز داده.
- *cache* ها در *CDN* ها و ISP ها و شبکه‌های سیار *mobile*
- *application – specific* در ISP ها و *CDN* ها

## مدل ساعت شنی

همینطور که قبلاً هزاربار گفتیم، لایه شبکه با یک پروتکل  $IP = Internet\ protocol$  کار میکند و لایه‌های دورتر با پروتکل‌های بیشتر (لایه‌های ۱ و ۵ پروتکل‌های متنوع‌تری از لایه‌های ۲ و ۴ دارند) این سادگی برای *connectivity* است و اینکه بخواهند توسط میلیارد ها دیوایس پیاده سازی شود و پیچیدگی را سمت *edge* آوردند.

البته به دلیل اضافه شدن *middle box* ها یک سری پروتکل مربوط به NFV و *caching* و *load balancing* و *firewall* و غیره هم اضافه شده است به آن. که در *core* شبکه *operate* میکنند.

البته بحث شده که *intelligence* کجای سیستم باشد؟

- بعضی‌ها قبلاً میگفتند درون شبکه باشد. مثلاً در شبکه‌های تلفنی سنتی (در هسته شبکه سویچ‌های هوشمند و کامپیوتری بود و در لبه همان دستگاه‌های تلفن بودند که هوشمند نبودند)
- بعداً گفتند که در *edge* شبکه باشد و به صورت *end to end* باشد. دارای *host*‌های کامپیوتری با پردازنده خوب.
- جدیداً این ایده را تقویت کرده و یک سری محاسبات هم به صورت *fog computing* در *edge* شبکه انجام میدهند. درواقع مفهوم *edge computing* اضافه شد و مراکز بزرگی که محاسبات بسیار بزرگتری نسبت به کامپیوترهای شخصی انجام میدهند در اینجا قرار میگیرند.

## مراجع

[1] “ch4-v8-slide-62-to-80,” <https://drive.google.com/drive/folders/>, 2022.